

Informe Laboratorio 4

Sección x

Alumno x

e-mail: alumno.contacto@mail.udp.cl

Junio de 2025

Índice

1. Descripción de actividades	2
2. Desarrollo de actividades según criterio de rúbrica	3
2.1. Investiga y documenta los tamaños de clave e IV	3
2.2. Solicita datos de entrada desde la terminal	5
2.3. Valida y ajusta la clave según el algoritmo	6
2.4. Implementa el cifrado y descifrado en modo CBC	7
2.5. Compara los resultados con un servicio de cifrado online	9
2.6. Describe la aplicabilidad del cifrado simétrico en la vida real	11

1. Descripción de actividades

Desarrollar un programa en Python utilizando la librería `pycryptodome` para cifrar y descifrar mensajes con los algoritmos DES, AES-256 y 3DES, permitiendo la entrada de la key, vector de inicialización y el texto a cifrar desde la terminal.

Instrucciones:

1. Investigación

- Investigue y documente el tamaño en bytes de la clave y el vector de inicialización (IV) requeridos para los algoritmos DES, AES-256 y 3DES. Mencione las principales diferencias entre cada algoritmo, sea breve.

2. El programa debe solicitar al usuario los siguientes datos desde la terminal

- Key correspondiente a cada algoritmo.
- Vector de Inicialización (IV) para cada algoritmo.
- Texto a cifrar.

3. Validación y ajuste de la clave

- Si la clave ingresada es menor que el tamaño necesario para el algoritmo complete los bytes faltantes agregando bytes adicionales generados de manera aleatoria (utiliza `get_random_bytes`).
- Si la clave ingresada es mayor que el tamaño requerido, trunque la clave a la longitud necesaria.
- Imprima la clave final utilizada para cada algoritmo después de los ajustes.

4. Cifrado y Descifrado

- Implemente una función para cada algoritmo de cifrado y descifrado (DES, AES-256, y 3DES). Use el modo CBC para todos los algoritmos.
- Asegúrese de utilizar el IV proporcionado por el usuario para el proceso de cifrado y descifrado.
- Imprima tanto el texto cifrado como el texto descifrado.

5. Comparación con un servicio de cifrado online

- Selecciona uno de los tres algoritmos (DES, AES-256 o 3DES), ingrese el mismo texto, key y vector de inicialización en una página web de cifrado online.
- Compare los resultados de tu programa con los del servicio online. Valide si el resultado es el mismo y fundamente su respuesta.

6. Aplicabilidad en la vida real

- Describa un caso, situación o problema donde usaría cifrado simétrico. Defina que algoritmo de cifrado simétrico recomendaría justificando su respuesta.
- Suponga que la recomendación que usted entregó no fue bien percibida por su contraparte y le pide implementar hashes en vez de cifrado simétrico. Argumente cuál sería su respuesta frente a dicha solicitud.

2. Desarrollo de actividades según criterio de rúbrica

2.1. Investiga y documenta los tamaños de clave e IV

Los algoritmos de cifrado simétrico DES, AES-256 y 3DES requieren tamaños específicos de clave y vector de inicialización (IV) para operar correctamente. A continuación se documentan estos valores según las especificaciones del NIST.

DES (Data Encryption Standard)

Según la especificación FIPS 46-3 [1], DES utiliza una clave de 64 bits (8 bytes), de los cuales solo 56 bits son efectivos ya que los 8 bits restantes se usan para paridad. El tamaño de bloque es de 64 bits, por lo que el IV en modo CBC debe ser de 8 bytes [4].

AES-256 (Advanced Encryption Standard)

El estándar FIPS 197 [2] define que AES-256 utiliza una clave de 256 bits (32 bytes). El tamaño de bloque de AES es fijo en 128 bits (16 bytes) para todas las variantes (128, 192 y 256 bits de clave). En modo CBC, el IV debe coincidir con el tamaño de bloque, es decir, 16 bytes [4].

3DES (Triple DES)

Según NIST SP 800-67 [3], 3DES aplica el algoritmo DES tres veces consecutivas. La clave total es de 192 bits (24 bytes), compuesta por tres claves DES de 64 bits cada una (con 56 bits efectivos por clave, dando 168 bits de seguridad nominal). El tamaño de bloque es el mismo que DES: 64 bits, por lo que el IV en modo CBC es de 8 bytes [4].

Principales diferencias

- **Seguridad:** DES es considerado inseguro desde hace años debido a su clave corta de 56 bits. 3DES mejoró la seguridad aplicando DES tres veces, pero también fue retirado por NIST en 2024. AES-256 es el estándar vigente recomendado por NIST [2].
- **Rendimiento:** AES es más rápido que 3DES, ya que 3DES ejecuta DES tres veces sobre cada bloque. Además, AES fue diseñado para ser eficiente tanto en hardware como en software [2].

- **Tamaño de bloque:** DES y 3DES operan con bloques de 64 bits, mientras que AES trabaja con bloques de 128 bits, lo que le da mayor resistencia contra ciertos ataques [4].

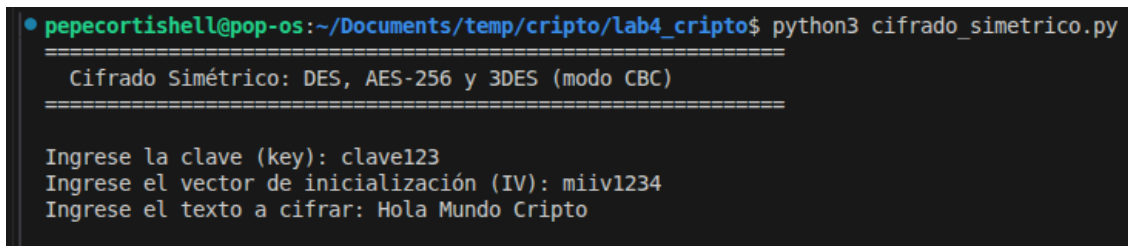
2.2. Solicita datos de entrada desde la terminal

El programa solicita tres datos al usuario mediante la función `input()` de Python: la clave (key), el vector de inicialización (IV) y el texto a cifrar. Estos valores se ingresan como cadenas de texto y luego se convierten a bytes con codificación UTF-8 para ser procesados por los algoritmos.

El siguiente fragmento muestra cómo se solicitan los datos:

```
1 key_input = input("\nIngrese la clave (key): ")
2 iv_input  = input("Ingrese el vector de inicializacion (IV): ")
3 texto     = input("Ingrese el texto a cifrar: ")
4
5 key_bytes  = key_input.encode("utf-8")
6 iv_bytes   = iv_input.encode("utf-8")
7 texto_bytes = texto.encode("utf-8")
```

La Figura 1 muestra la ejecución del programa en la terminal donde se ingresan los tres valores solicitados.



```
● pepecortishell@pop-os:~/Documents/temp/cripto/lab4_cripto$ python3 cifrado_simetrico.py
=====
Cifrado Simétrico: DES, AES-256 y 3DES (modo CBC)
=====

Ingrese la clave (key): clave123
Ingrese el vector de inicialización (IV): miiv1234
Ingrese el texto a cifrar: Hola Mundo Cripto
```

Figura 1: Ingreso de datos desde la terminal: clave, IV y texto a cifrar.

2.3. Valida y ajusta la clave según el algoritmo

El programa implementa una función llamada `ajustar_clave()` que recibe la clave ingresada, el tamaño requerido por el algoritmo y el nombre del algoritmo. La lógica es la siguiente:

- Si la clave es **menor** al tamaño requerido, se completan los bytes faltantes con bytes aleatorios generados mediante `get_random_bytes()`.
- Si la clave es **mayor** al tamaño requerido, se trunca a la longitud necesaria.
- Si la clave tiene el **tamaño exacto**, se usa tal cual.

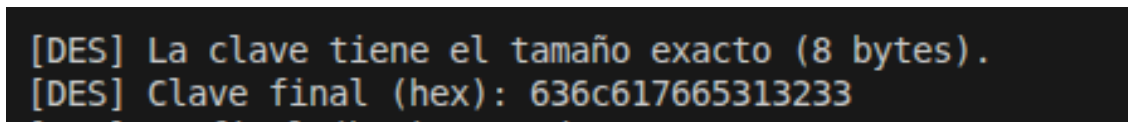
El fragmento de código correspondiente es:

```

1 def ajustar_clave(key_bytes, tam_requerido, nombre_algo):
2     if len(key_bytes) < tam_requerido:
3         relleno = get_random_bytes(tam_requerido - len(key_bytes))
4         key_bytes = key_bytes + relleno
5     elif len(key_bytes) > tam_requerido:
6         key_bytes = key_bytes[:tam_requerido]
7     return key_bytes

```

Después del ajuste, se imprime la clave final en formato hexadecimal para cada algoritmo. Las Figuras 2 y 3 muestran los dos tipos de respuesta que genera el programa según el tamaño de la clave ingresada.

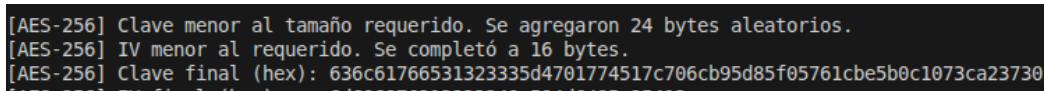


```

[DES] La clave tiene el tamaño exacto (8 bytes).
[DES] Clave final (hex): 636c617665313233

```

Figura 2: Caso donde la clave ingresada tiene el tamaño exacto requerido por el algoritmo.



```

[AES-256] Clave menor al tamaño requerido. Se agregaron 24 bytes aleatorios.
[AES-256] IV menor al requerido. Se completó a 16 bytes.
[AES-256] Clave final (hex): 636c61766531323335d4701774517c706cb95d85f05761cbe5b0c1073ca23730

```

Figura 3: Caso donde la clave es menor al tamaño requerido y se completa con bytes aleatorios.

2.4. Implementa el cifrado y descifrado en modo CBC

Se implementaron funciones separadas para cifrar y descifrar con cada algoritmo (DES, AES-256 y 3DES), todas utilizando el modo CBC. Se utiliza padding PKCS7 mediante las funciones `pad()` y `unpad()` de `pycryptodome`, ya que CBC requiere que el texto sea múltiplo del tamaño de bloque.

A continuación se muestra como ejemplo la función de cifrado y descifrado para AES-256:

```

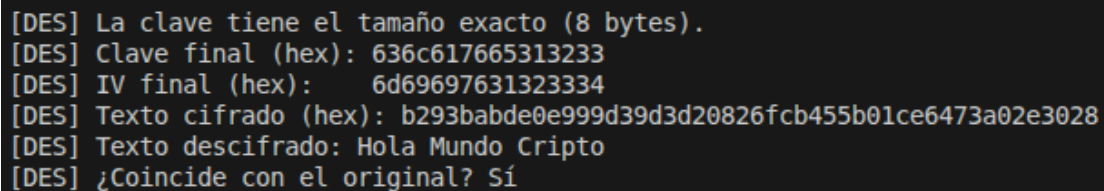
1 def cifrar_aes256(texto, key, iv):
2     cipher = AES.new(key, AES.MODE_CBC, iv)
3     ct = cipher.encrypt(pad(texto, AES.block_size))
4     return ct
5
6 def descifrar_aes256(ct, key, iv):
7     cipher = AES.new(key, AES.MODE_CBC, iv)
8     pt = unpad(cipher.decrypt(ct), AES.block_size)
9     return pt

```

Las funciones para DES y 3DES siguen la misma estructura, cambiando únicamente el módulo (DES, DES3) y el tamaño de bloque correspondiente.

El programa imprime para cada algoritmo: la clave final utilizada, el IV, el texto cifrado en hexadecimal, el texto descifrado y una verificación de que el texto descifrado coincide con el original.

La Figura 4 muestra los resultados del cifrado y descifrado con DES.



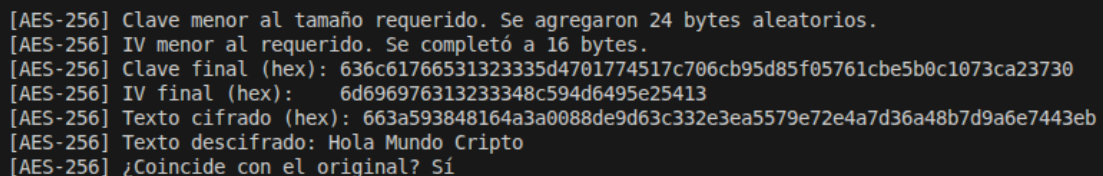
```

[DES] La clave tiene el tamaño exacto (8 bytes).
[DES] Clave final (hex): 636c617665313233
[DES] IV final (hex): 6d69697631323334
[DES] Texto cifrado (hex): b293babde0e999d39d3d20826fcb455b01ce6473a02e3028
[DES] Texto descifrado: Hola Mundo Cripto
[DES] ¿Coincide con el original? Sí

```

Figura 4: Resultado del cifrado y descifrado con DES en modo CBC.

La Figura 5 muestra los resultados para AES-256.



```

[AES-256] Clave menor al tamaño requerido. Se agregaron 24 bytes aleatorios.
[AES-256] IV menor al requerido. Se completó a 16 bytes.
[AES-256] Clave final (hex): 636c61766531323335d4701774517c706cb95d85f05761cbe5b0c1073ca23730
[AES-256] IV final (hex): 6d696976313233348c594d6495e25413
[AES-256] Texto cifrado (hex): 663a593848164a3a0088de9d63c332e3ea5579e72e4a7d36a48b7d9a6e7443eb
[AES-256] Texto descifrado: Hola Mundo Cripto
[AES-256] ¿Coincide con el original? Sí

```

Figura 5: Resultado del cifrado y descifrado con AES-256 en modo CBC.

La Figura 6 muestra los resultados para 3DES.

```
[3DES] Clave menor al tamaño requerido. Se agregaron 16 bytes aleatorios.
[3DES] Clave final (hex): 636c617665313233ba3caf9781c8ced167b8a57f62b87dad
[3DES] IV final (hex): 6d69697631323334
[3DES] Texto cifrado (hex): 3e91050be38a92b4ea8c9866321680719a5f409c00aa62a9
[3DES] Texto descifrado: Hola Mundo Cripto
[3DES] ¿Coincide con el original? Sí
```

Figura 6: Resultado del cifrado y descifrado con 3DES en modo CBC.

Finalmente, la Figura 7 muestra una ejecución completa del programa con los tres algoritmos, donde se verifica que el texto descifrado coincide con el original en todos los casos.

```
pepecortishell@pop-os:~/Documents/temp/cripto/lab4_cripto$ python3 cifrado_simetrico.py
=====
Cifrado Simétrico: DES, AES-256 y 3DES (modo CBC)
=====

Ingrese la clave (key): clave123
Ingrese el vector de inicialización (IV): miiv1234
Ingrese el texto a cifrar: Hola Mundo Cripto

-----
Resultados
-----

[DES] La clave tiene el tamaño exacto (8 bytes).
[DES] Clave final (hex): 636c617665313233
[DES] IV final (hex): 6d69697631323334
[DES] Texto cifrado (hex): b293babde0e999d39d3d20826fcb455b01ce6473a02e3028
[DES] Texto descifrado: Hola Mundo Cripto
[DES] ¿Coincide con el original? Sí

[AES-256] Clave menor al tamaño requerido. Se agregaron 24 bytes aleatorios.
[AES-256] IV menor al requerido. Se completó a 16 bytes.
[AES-256] Clave final (hex): 636c61766531323335d4701774517c706cb95d85f05761cbe5b0c1073ca23730
[AES-256] IV final (hex): 6d696976313233348c594d6495e25413
[AES-256] Texto cifrado (hex): 663a593848164a3a0088de9d63c332e3ea5579e72e4a7d36a48b7d9a6e7443eb
[AES-256] Texto descifrado: Hola Mundo Cripto
[AES-256] ¿Coincide con el original? Sí

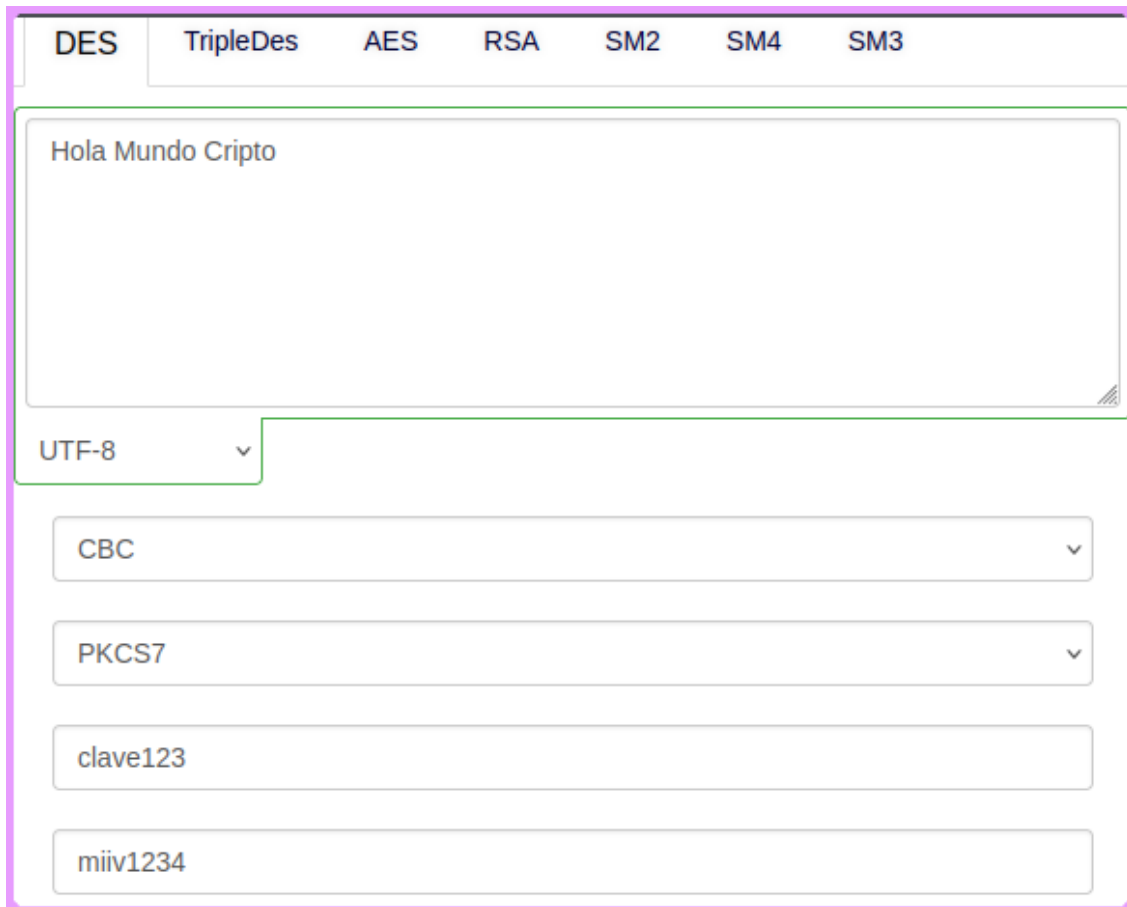
[3DES] Clave menor al tamaño requerido. Se agregaron 16 bytes aleatorios.
[3DES] Clave final (hex): 636c617665313233ba3caf9781c8ced167b8a57f62b87dad
[3DES] IV final (hex): 6d69697631323334
[3DES] Texto cifrado (hex): 3e91050be38a92b4ea8c9866321680719a5f409c00aa62a9
[3DES] Texto descifrado: Hola Mundo Cripto
[3DES] ¿Coincide con el original? Sí
```

Figura 7: Ejecución completa del programa mostrando los tres algoritmos.

2.5. Compara los resultados con un servicio de cifrado online

Para validar los resultados del programa, se seleccionó el algoritmo DES y se utilizó la herramienta online The-X Cryptography [6]. Se ingresaron los mismos parámetros: clave `clave123`, IV `miiv1234`, texto `Hola Mundo Cripto`, modo CBC y padding PKCS7.

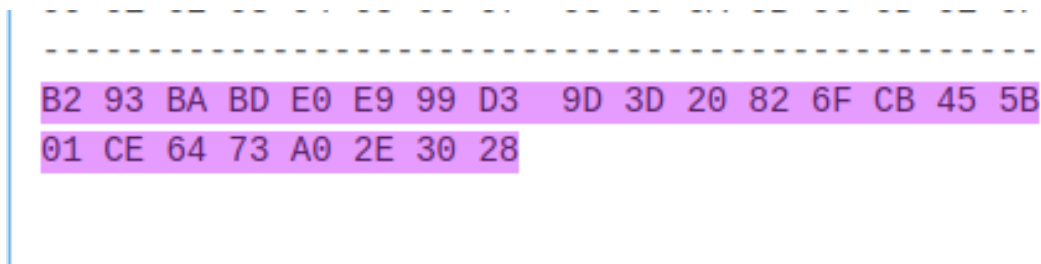
La Figura 8 muestra la configuración utilizada en la herramienta online.



The screenshot shows the configuration interface of the The-X Cryptography online tool. At the top, there are tabs for different encryption algorithms: DES, TripleDes, AES, RSA, SM2, SM4, and SM3. The 'DES' tab is selected. Below the tabs, there is a large text input field containing the text 'Hola Mundo Cripto'. Underneath this field, there is a dropdown menu for the character encoding, which is set to 'UTF-8'. Below the encoding dropdown, there are two more dropdown menus: one for the encryption mode, set to 'CBC', and another for the padding scheme, set to 'PKCS7'. At the bottom, there are two text input fields: the first contains the key 'clave123' and the second contains the IV 'miiv1234'.

Figura 8: Configuración del cifrado DES-CBC en la herramienta online [6].

La Figura 9 muestra el resultado obtenido por el servicio online.



The screenshot shows the result of the online encryption. It displays two lines of hexadecimal data on a purple background. The first line contains the characters: B2 93 BA BD E0 E9 99 D3 9D 3D 20 82 6F CB 45 5B. The second line contains the characters: 01 CE 64 73 A0 2E 30 28. Above the first line, there are two dashed lines, one solid and one dashed, indicating a header or separator.

Figura 9: Resultado del cifrado online con los mismos parámetros.

Para que ambos resultados coincidan, es necesario que se utilicen exactamente los mismos parámetros: algoritmo, modo de operación, clave, IV y padding. Si alguno difiere, el texto cifrado será completamente distinto. En caso de no coincidir, la causa más común es una diferencia en la codificación de la clave o el IV (por ejemplo, texto UTF-8 versus hexadecimal) o en el tipo de padding.

2.6. Describe la aplicabilidad del cifrado simétrico en la vida real

Caso de uso: Protección de registros médicos

Un escenario real donde se aplica cifrado simétrico es la protección de fichas médicas en centros de salud. La normativa HIPAA del Departamento de Salud de Estados Unidos establece que los datos electrónicos de pacientes deben ser cifrados para considerarse protegidos ante una filtración [5]. El estándar recomendado para datos en reposo es AES-256, ya que es el algoritmo vigente aprobado por NIST, ofrece una clave de 256 bits resistente a fuerza bruta y tiene buen rendimiento para archivos grandes. DES y 3DES no son opciones viables porque ya fueron retirados por NIST.

El funcionamiento sería sencillo: cuando se guarda una ficha, el sistema la cifra con AES-256 usando una clave almacenada de forma segura. Cuando un médico necesita leerla, el sistema la descifra temporalmente.

Respuesta ante la solicitud de usar hashes

No sería posible reemplazar cifrado simétrico por hashing. Un hash es una operación de una sola vía que genera un resumen fijo a partir de un dato, pero no permite recuperar el dato original. Sirve para verificar integridad o almacenar contraseñas, no para proteger información que necesita ser leída después. El cifrado simétrico sí es reversible: se cifra con una clave y se descifra con la misma. En el caso de las fichas médicas, los médicos necesitan leer el contenido, así que es obligatorio poder descifrar. Si se aplicara un hash, los datos se perderían.

Conclusiones y comentarios

En este laboratorio se implementó un programa en Python capaz de cifrar y descifrar mensajes con DES, AES-256 y 3DES en modo CBC. Se pudo comprobar que los tres algoritmos producen un texto cifrado que, al ser descifrado con la misma clave e IV, devuelve el texto original sin alteraciones. Además, se validó el ajuste automático de claves cuando el usuario ingresa una clave de tamaño incorrecto, lo cual es un aspecto importante en la práctica.

La comparación con un servicio online permitió verificar que, si se utilizan los mismos parámetros (clave, IV, modo y padding), los resultados deben coincidir. Esto refuerza que los algoritmos de cifrado son determinísticos: con las mismas entradas, siempre producen la misma salida. También quedó claro que AES-256 es el algoritmo más recomendable actualmente, ya que DES y 3DES fueron retirados por NIST debido a vulnerabilidades en el tamaño de clave y bloque.

Referencias

- [1] National Institute of Standards and Technology. (1999). *Data Encryption Standard (DES)* (FIPS PUB 46-3). U.S. Department of Commerce. <https://csrc.nist.gov/publications/detail/fips/46/3/archive/1999-10-25>
- [2] National Institute of Standards and Technology. (2001). *Advanced Encryption Standard (AES)* (FIPS PUB 197). U.S. Department of Commerce. <https://csrc.nist.gov/publications/detail/fips/197/final>
- [3] Barker, W. y Barker, E. (2012). *Recommendation for the Triple Data Encryption Algorithm (TDEA) Block Cipher* (NIST SP 800-67 Rev. 2). National Institute of Standards and Technology. <https://csrc.nist.gov/publications/detail/sp/800-67/rev-2/final>
- [4] Dworkin, M. (2001). *Recommendation for Block Cipher Modes of Operation: Methods and Techniques* (NIST SP 800-38A). National Institute of Standards and Technology. <https://csrc.nist.gov/publications/detail/sp/800-38a/final>
- [5] U.S. Department of Health and Human Services. (2013). *HIPAA Security Rule: Technical Safeguards* (45 CFR §164.312). <https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html>
- [6] The-X.cn. (s.f.). *DES Encrypt/Decrypt Online Tool*. Recuperado el 29 de mayo de 2025, de <https://the-x.cn/en-us/cryptography/Des.aspx>